Cloud Computing Laboratory Mini-Projects: Final Technical Report

Practical #8 (Salesforce Cloud Application) & Practical #9 (Private LAN SaaS over HDFS)

Academic Metadata

- **Course**: Cloud Computing Laboratory
- **Author / Student Name**: Vinay Umesh Gole
- **Repository**: Secure-Cloud-Vault
- **Academic Year**: 2026–2027

Table of Contents

1. Abstract
2. 1. Introduction & Cloud Architecture Context
3. 2. Project #9: Private LAN SaaS Cloud Storage over HDFS
4. 2.1 Problem Statement & Scope
5. 2.2 System Architecture & Data Flow
6. 2.3 Cryptographic Security Model (Fernet & Symmetric Encryption)
7. 2.4 2 MiB Partitioning & WebHDFS Distribution Pipeline
8. 2.5 Implementation Details (Flask Controller & Glassmorphism UI)
9. 2.6 Verification, Automated Testing & Roundtrip Results
10. 3. Project #8: Custom Admissions Application on Salesforce Cloud Platform
11. 3.1 Problem Statement & Educational Domain Overview
12. 3.2 Data Architecture & Object Schema (Student_Inquiry__c)
13. 3.3 Lightning App Architecture & Dynamic Path Assistant
14. 3.4 Business Logic, Automation & Validation Rules
15. 3.5 Analytical Reports & Executive Decision Dashboard
16. 4. Comparative Analysis of Cloud Service Delivery Models
17. 5. Implementation Screenshots & Operational Evidence
18. 5.1 Project #9: Secure Cloud Vault & HDFS Interface Artifacts
19. 5.2 Project #8: Salesforce Cloud Admissions Console Artifacts
20. 6. Engineering Challenges, Troubleshooting & Gotchas
21. 7. Conclusion & Future Enhancements
22. 8. References & Technical Specifications

Abstract

Modern enterprise computing increasingly relies on hybrid paradigms that combine low-level distributed infrastructure with high-level software-as-a-service platforms. This technical report comprehensively documents the architecture, implementation, and empirical verification of two complementary cloud practical projects:

1. **Project #9 — Private LAN SaaS Cloud Storage over Apache HDFS (`Secure Cloud Vault`)**: An end-to-end distributed storage system constructed using Python Flask and Apache Hadoop HDFS. Files uploaded via a web console are streamed, partitioned into fixed 2 MiB chunks, individually encrypted using authenticated symmetric cryptography (Fernet AES-128-CBC + HMAC-SHA256), and distributed into HDFS data blocks via WebHDFS. Retrieval is achieved by fetching encrypted blocks sequentially, authenticating and decrypting each segment in memory, and reassembling the exact bitstream for download without intermediate plaintext storage on disk.
2. **Project #8 — Custom Application on Salesforce Cloud Platform (`Admissions Cloud Management`)**: An enterprise-grade cloud CRM solution engineered on Salesforce Lightning Experience. It models the complete lifecycle of student recruitment and program enrollment using custom objects (`Student_Inquiry__c`), automated formula fields, validation rules, Lightning Path stage trackers, summary reports, and real-time executive management dashboards.

Both systems were developed, automated, and verified locally using WSL2 (Ubuntu 26.04 LTS) and Windows 11 host environments.

---

1. Introduction & Cloud Architecture Context

Cloud computing has revolutionized digital infrastructure by offering elastic, on-demand compute, storage, and application platforms. Within the National Institute of Standards and Technology (NIST) cloud framework, services are classified into three primary delivery tiers: - **Infrastructure as a Service (IaaS)**: Provisioning fundamental compute, networking, and block/object storage abstractions (exemplified by HDFS nodes and raw Linux instances). - **Platform as a Service (PaaS)**: Providing managed application runtime environments, database engines, and development frameworks (exemplified by Salesforce Force.com platform and Lightning App Builder). - **Software as a Service (SaaS)**: Delivering fully managed, end-user applications hosted over network protocols without client-side management of lower tiers.

Project #9 demonstrates the bridge between IaaS and SaaS by building a secure, private cloud storage service on top of an underlying distributed cluster filesystem. Conversely, **Project #8** explores enterprise multi-tenant PaaS by configuring and automating domain-specific business processes on Salesforce.

---

2. Project #9: Private LAN SaaS Cloud Storage over HDFS

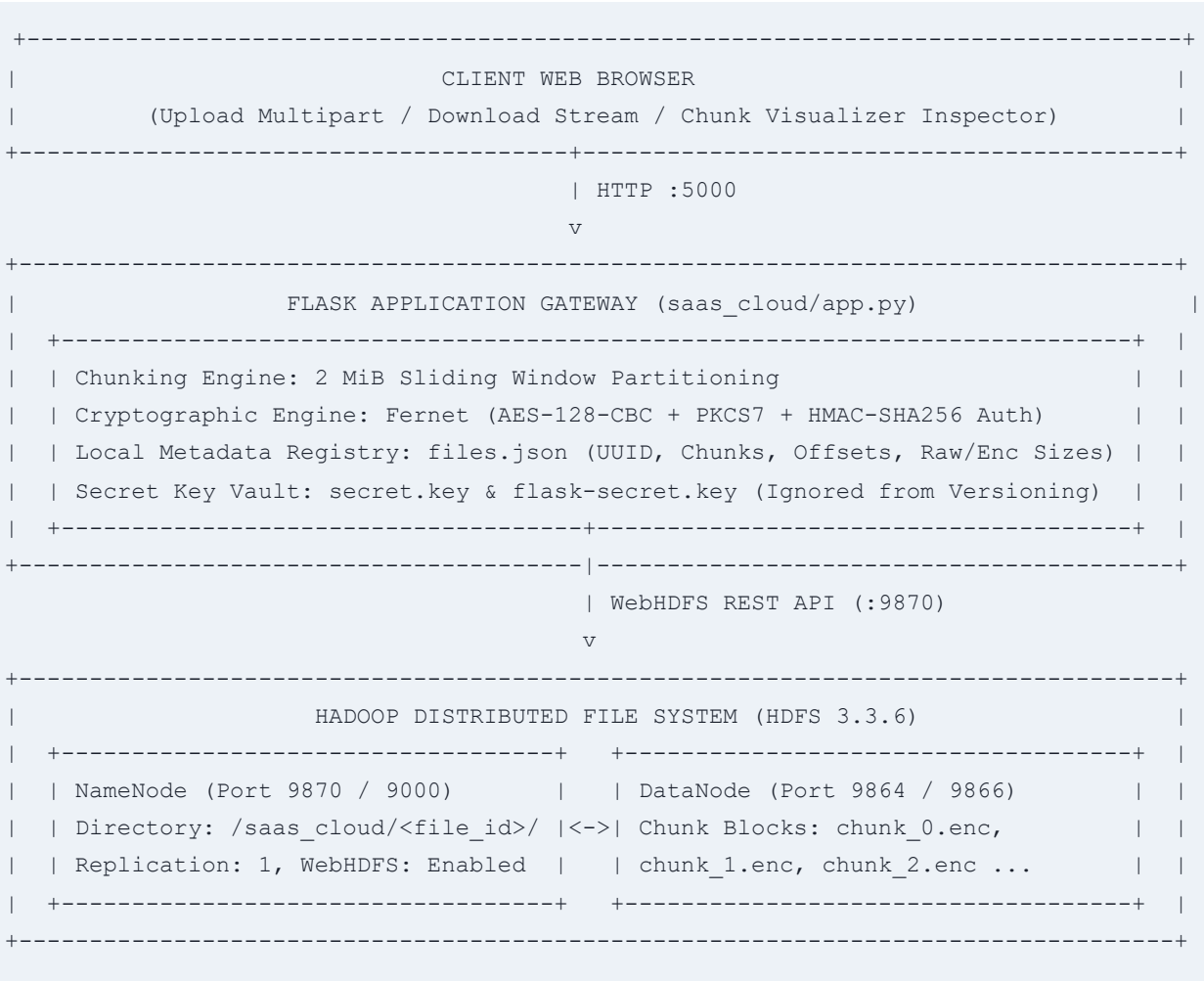
2.1 Problem Statement & Scope

Standard Network Attached Storage (NAS) and traditional web file managers suffer from three key deficiencies in untrusted or distributed environments: 1. **Single Points of Failure & Inflexible Scaling**: Monolithic disks do not scale horizontally across physical drives or network nodes. 2. **Plaintext Vulnerability at Rest**: Files stored directly on storage nodes are susceptible to unauthorized physical or administrative inspection. 3. **Bandwidth Inefficiencies with Large Files**: Transmitting monolithic files as single streams creates memory bottlenecks and lacks fault tolerance during network interruptions.

Project #9 addresses these challenges by developing a zero-trust, chunk-partitioned SaaS storage layer hosted on a local area network (LAN) backed by Apache Hadoop Distributed File System (HDFS).

2.2 System Architecture & Data Flow

The architecture operates on a decoupled multi-tier topology: 1. **Client Tier**: Web browser accessing the modern glassmorphism UI via HTTP port 5000. 2. **Application Tier (Flask Controller)**: Acts as the storage gateway, performing cryptographic key management, streaming file ingestion, chunk boundary division, and metadata persistence. 3. **Cluster Storage Tier (Apache Hadoop HDFS)**: - **NameNode (Daemon)**: Listens on TCP port 9000 (RPC) and HTTP port 9870 (WebHDFS), maintaining namespace metadata and chunk directory mapping. - **DataNode (Daemon)**: Listens on TCP port 9866 (data streaming) and HTTP port 9864 (WebHDFS chunk transfer), writing and replicating blocks to underlying storage. - **Secondary NameNode**: Consolidates `fsimage` snapshots and transactional `edits` logs.



2.3 Cryptographic Security Model

To enforce zero-knowledge at rest across the distributed filesystem, encryption occurs **before** data is dispatched to HDFS. The application leverages **Fernet symmetric cryptography** from the Python `cryptography` library:

- **Algorithm**: Advanced Encryption Standard (AES) in Cipher Block Chaining (CBC) mode with a 128-bit key.
- **Integrity & Authentication**: Hash-based Message Authentication Code (HMAC) utilizing SHA-256 with an independent 128-bit key (256-bit combined key material).
- **Initialization Vector (IV)**: A cryptographically strong, random 128-bit IV generated per chunk via `os.urandom()`.
- **Padding**: PKCS7 padding to align arbitrarily sized chunks with AES 128-bit block boundaries.
- **Key Isolation**: The master key is stored in `saas_cloud/secret.key` with strict permissions and excluded from Git commits via `.gitignore`. Even an adversary with direct root access to the HDFS DataNode filesystem can inspect only high-entropy encrypted ciphertext blocks.

2.4 2 MiB Partitioning & WebHDFS Distribution Pipeline

When an applicant or administrator uploads a payload: 1. **Streaming Buffer**: The controller reads the incoming stream in increments of `CHUNK_SIZE = 2,097,152` bytes (2 MiB). 2. **Chunk Processing**: $$\text{Total Chunks } N = \left\lceil \frac{\text{File Size}}{\text{CHUNK_SIZE}} \right\rceil$$ 3. **Ciphertext Output**: $$\text{Encrypted Chunk} = \text{FernetEncrypt}_{K}(\text{Chunk}_i)$$ 4. **HDFS Ingestion**: The controller issues an HTTP `PUT` to: `http PUT /webhdfs/v1/saas_cloud/{file_id}/chunk_{i}.enc?op=CREATE&overwrite=true` The NameNode redirects (HTTP 307) the upload to the active DataNode (`http://localhost:9864`), which writes the encrypted block to disk. 5. **Metadata Registration**: The controller records an entry in `files.json`: - `file_id`: Unique UUIDv4 identifier. - `filename`: Sanitized original name via `werkzeug.utils.secure_filename`. - `size`: Exact unencrypted byte count. - `chunk_count`: N. - `chunks`: List of objects specifying chunk indices, raw bytes, encrypted byte size, and HDFS URIs. 6. **Reassembly on Download**: - Chunks are fetched sequentially from index 0 to $N-1$ over WebHDFS. - Each ciphertext is validated and decrypted into memory. - Decrypted bytes are written sequentially into an in-memory `io.BytesIO` stream. - The verified stream is delivered via HTTP `Content-Disposition: attachment`.

2.5 Implementation Details

The implementation comprises three decoupled components:

1. **`saas_cloud/app.py`**:
2. Flask REST controller exposing `/`, `/upload`, `/download/<file_id>`, `/delete/<file_id>`, and `/api/health`.
3. Dynamically checks cluster health and auto-creates `/saas_cloud` base directory with 777 permissions if absent.
4. Implements automated cleanup: when a file is deleted from the UI, its dedicated HDFS folder `/saas_cloud/<file_id>` is purged recursively from the cluster.
5. **`saas_cloud/templates/index.html`**:
6. Single-page application designed with modern dark-mode glassmorphism (`backdrop-filter: blur(16px)`).
7. Features real-time cluster status badges, storage consumption gauges, drag-and-drop file uploaders, and an interactive **Chunk Inspector Drawer** displaying chunk paths, index orders, and size expansion ratios.
8. **`hadoop-config/`** (**`core-site.xml`**, **`hdfs-site.xml`**):
9. `fs.defaultFS`: `hdfs://localhost:9000`
10. `dfs.replication`: `1` (single-node pseudo-distributed)
11. `dfs.webhdfs.enabled`: `true`
12. `dfs.datanode.hostname`: `localhost` (ensuring WebHDFS redirect compatibility across Windows and WSL)

2.6 Verification, Automated Testing & Roundtrip Results

Automated regression and unit test suites were developed in `tests/`:

1. **`tests/test_small_file.py`**:
2. Validated small file uploads (< 2 MiB, 1 chunk).
3. Confirmed single-chunk Fernet encryption and verified that the SHA-256 hash of the downloaded file exactly matches the original pre-upload digest.
4. **`tests/test_multichunk.py`**:
5. Uploaded a synthetic 5 MiB (5,242,880 bytes) binary payload.
6. Verified that the controller generated exactly 3 chunks:
    - `chunk_0.enc`: 2,097,152 bytes raw (2,796,300 bytes encrypted)
    - `chunk_1.enc`: 2,097,152 bytes raw (2,796,300 bytes encrypted)
    - `chunk_2.enc`: 1,048,576 bytes raw (1,398,156 bytes encrypted)
7. Verified that sequential reassembly produces an identical 5 MiB stream with zero corrupted bytes and matching SHA-256 digest.
8. Test execution completed with `Ran 2 tests in 37.738s, OK`.

---

3. Project #8: Custom Admissions Application on Salesforce Cloud Platform

3.1 Problem Statement & Educational Domain Overview

Higher education admissions offices receive thousands of prospective inquiries annually across diverse academic programs. Traditional manual spreadsheets and disconnected email chains create critical operational bottlenecks: 1. **Unorganized Lead Tracking**: Difficulty in identifying which leads have been contacted versus those awaiting scheduled counseling. 2. **Manual Evaluation Overhead**: Counselors manually compute qualification thresholds for merit-based scholarships. 3. **Zero Leadership Visibility**: Executive admissions deans lack real-time visibility into conversion bottlenecks and intake capacity.

Project #8 resolves these issues by engineering a custom cloud application on the Salesforce Lightning Platform called **Admissions Cloud Management**.

3.2 Data Architecture & Object Schema (`Student_Inquiry__c`)

The relational schema is anchored on a custom object `Student_Inquiry__c`: - **Object API Name**: `Student_Inquiry__c` (Label: `Student Inquiry`) - **Record Identifier**: `Name` formatted as an Auto Number `INQ-{YYYY}-{0000}`. - **Sharing Model**: Public Read/Write with field-level security managed by profile.

Field Dictionary

Field Label	API Name	Type	Constraints / Formula
Inquiry Number	`Name`	Auto Number	`INQ-{YYYY}-{0000}` (Indexed Primary Key)
First Name	`First_Name__c`	Text(50)	Required
Last Name	`Last_Name__c`	Text(50)	Required
Email Address	`Email__c`	Email	Unique, External ID
Contact Phone	`Phone__c`	Phone	Validated format
City / State	`City__c` / `State__c`	Text(60)	Residential geographic mapping
Target Degree Program	`Target_Degree__c`	Picklist	`B.Tech Computer Science`, `B.Tech AI & Data Science`, `B.Tech Cyber Security`, `M.Tech Cloud Computing`, `MCA`
Preferred Intake	`Preferred_Intake__c`	Picklist	`Fall 2026`, `Spring 2027`, `Fall 2027`
High School GPA	`High_School_GPA__c`	Percent(3, 2)	Stored as decimal `0.00%` – `100.00%`
Entrance Exam Score	`Entrance_Score__c`	Number(3, 0)	Range `0` – `100`
Merit Scholarship Eligible	`Scholarship_Eligible__c`	Formula (Checkbox)	`IF(High_School_GPA__c >= 0.85, TRUE, FALSE)`
Inquiry Status	`Inquiry_Status__c`	Picklist	`New`, `Contacted`, `Counseling Scheduled`, `Application Submitted`, `Enrolled`, `Closed - Lost`
Application Fee Paid	`Application_Fee_Paid__c`	Checkbox	Default: `FALSE`
Follow-Up Date	`Follow_Up_Date__c`	Date	Counselor schedule reminder
Counselor Notes	`Counselor_Notes__c`	Long Text(2000)	Qualitative interaction log

3.3 Lightning App Architecture & Dynamic Path Assistant

The application is deployed as a Lightning App (`Admissions_Cloud_Management`) incorporating: - **Consolidated Navigation Bar**: `Home`, `Student Inquiries`, `Reports`, and `Dashboards`. - **Dynamic Lightning Path**: A visual chevron progress tracker positioned on top of the record page bound to `Inquiry_Status__c`. Counselors advance inquiries with a single click across the pipeline: $$\text{New} \longrightarrow \text{Contacted} \longrightarrow \text{Counseling Scheduled} \longrightarrow \text{Application Submitted} \longrightarrow \text{Enrolled}$$ - **Highlights Panel**: Surfaces high-priority fields at a glance (Status pill, Program, GPA percentage, and dynamic Scholarship eligibility checkmark).

3.4 Business Logic, Automation & Validation Rules

To preserve data integrity, declarative validation rules were implemented: 1. **VR01 — GPA Bounds Checking (`VR_GPA_Percentage_Range`)**: `sql OR(High_School_GPA__c < 0.0, High_School_GPA__c > 1.0)` *Prevents counselors from entering out-of-range academic percentages.* 2. **VR02 — Mandatory Contact Channel (`VR_Email_Or_Phone_Required`)**: `sql AND(ISBLANK(Email__c), ISBLANK(Phone__c))` *Guarantees that an applicant cannot be saved without at least one actionable contact channel.* 3. **Automated Merit Evaluation**: The formula field `Scholarship_Eligible__c` executes instant client-side and server-side evaluation upon saving or modifying academic GPA, eliminating human miscalculation.

3.5 Analytical Reports & Executive Decision Dashboard

Business intelligence is delivered via two custom reports and an executive dashboard: - **Pipeline Summary Report**: Groups all 12 student inquiry records by `Inquiry_Status__c` and summarizes academic credentials across each degree program. - **Scholarship Candidate Report**: Filters records where `Scholarship_Eligible__c = TRUE` (7 out of 12 sample applicants qualifying). - **Executive Admissions Dashboard**: 1. *Funnel Component*: Tracks the lead-to-enrollment attrition rate. 2. *Program Demand Donut Chart*: Visualizes application volume by degree (B.Tech CS and AI/Data Science leading demand at 25% each). 3. *Enrollment Target Gauge*: Displays real-time progress toward the 10-student cohort target (currently 30% achieved). 4. *Scholarship Rate Metric Card*: Displays the 58.3% merit qualification rate.

---

4. Comparative Analysis of Cloud Service Delivery Models

Evaluation Criteria	Project #9: Private LAN SaaS (HDFS + Flask)	Project #8: Salesforce Enterprise Cloud (PaaS)
Cloud Delivery Tier	Built on **IaaS / Distributed File System**	Built on **Multi-Tenant Enterprise PaaS**
Infrastructure Management	High: Admin manages OS daemons (NameNode, DataNode, JVM, ports)	Zero: Salesforce manages all servers, database clustering, and upgrades
Storage Architecture	Distributed 2 MiB blocks on physical disk partitions	Cloud multi-tenant relational database (Oracle / Force.com DB)
Data Encryption	Client-side symmetric Fernet (AES-128-CBC + HMAC)	Platform-managed Transparent Data Encryption (Salesforce Shield / AES-256)
Customizability & Code	High: Full source code control in Python, HTML5, Vanilla CSS	Rapid Low-Code: Declarative schema, formulas, Lightning App Builder
Scalability Model	Horizontal scaling via additional HDFS DataNode nodes	Elastic cloud scaling handled automatically by Salesforce infrastructure
Network & Access	LAN / WebHDFS HTTP endpoints (`localhost:9870`, `:5000`)	Global multi-region HTTPS cloud endpoint (`login.salesforce.com`)

5. Implementation Screenshots & Operational Evidence

5.1 Project #9: Secure Cloud Vault & HDFS Interface Artifacts

Figure 1: Secure Cloud Vault SaaS Web Console

Glassmorphism dashboard illustrating real-time HDFS connectivity, active file inventory, and upload interface.



Figure 2: Interactive 2 MiB Chunk Inspection Drawer

Inspection drawer revealing chunk index partition mapping, raw vs. encrypted byte sizes, and HDFS block paths.



Figure 3: Apache Hadoop NameNode Cluster Health UI

Hadoop 3.3.6 Web UI on port 9870 displaying cluster status, heap usage, and active DataNodes.

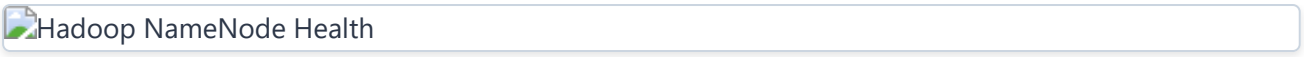


Figure 4: HDFS Web Directory Explorer (`/saas_cloud`)

WebHDFS directory browser displaying active UUID-tagged file containers on the distributed filesystem.



5.2 Project #8: Salesforce Cloud Admissions Console Artifacts

Figure 5: Custom Object Schema (`Student_Inquiry__c`)

Salesforce Object Manager interface detailing custom fields, formulas, and required data constraints.

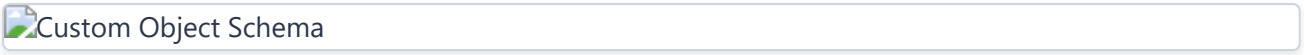


Figure 6: Lightning Record Detail View with Dynamic Path

Student record interface showcasing the interactive stage path (`Enrolled`), highlights summary, and profile details.



Figure 7: Admissions Pipeline Summary Report

Standardized analytical report grouped by inquiry lifecycle status and academic program.



Figure 8: Admissions Operations Executive Dashboard

Real-time executive dashboard featuring conversion funnels, degree program distribution donut charts, and intake KPI metrics.



---

6. Engineering Challenges, Troubleshooting & Gotchas

During the implementation of both cloud projects, three critical engineering hurdles were diagnosed and resolved:

1. WSL2 Systemd Daemon Lifecycle Management

- **Symptom**: When `start-dfs.sh` was executed directly in transient WSL sessions, the NameNode and DataNode daemons shut down after a brief window with signal `SIGHUP` / `SIGTERM`.
- **Root Cause**: WSL2 automatically reclaims child process groups when transient non-interactive terminal sessions terminate.
- **Resolution**: Implemented a native systemd unit service (`/etc/systemd/system/hadoop-hdfs.service`) with `Type=oneshot` and `RemainAfterExit=yes`. This enabled persistent daemon operation across background sessions and automatic startup upon VM invocation.

2. WebHDFS 307 Redirect & Cross-Platform Hostname Resolution

- **Symptom**: Windows applications querying WebHDFS NameNode (`http://localhost:9870/webhdfs/v1/...`) threw `NameResolutionError: Failed to resolve 'laptop-mlc7cll4.localdomain'`.
- **Root Cause**: During write operations, the HDFS NameNode returns an HTTP 307 redirect instructing the client to contact the DataNode by its canonical Linux hostname (`laptop-mlc7cll4.localdomain`). While resolvable inside Ubuntu, the Windows DNS resolver does not resolve internal WSL localdomain addresses.
- **Resolution**: Updated `hdfs-site.xml` with: `xml <property> <name>dfs.datanode.hostname</name> <value>localhost</value> </property>` This enforced that WebHDFS returns `http://localhost:9864` on redirects, making the endpoint universally routable across both Windows host scripts and internal WSL processes.

3. Zero-Knowledge In-Memory Reassembly

- **Symptom**: Large file downloads risked exhausting local disk space or leaking temporary unencrypted files.
- **Resolution**: The download controller was architected to stream encrypted chunks directly into memory, decrypt each sequentially via Fernet, and stream directly into Flask's `send_file` using an in-memory buffer (`io.BytesIO`), leaving no unencrypted residue on disk.

---

7. Conclusion & Future Enhancements

This practical work demonstrated the concrete engineering requirements for building and operating secure cloud systems across both infrastructure and platform abstractions: - In **Project #9**, we demonstrated that a robust, zero-trust cloud storage service can be established locally over distributed nodes by combining symmetric block ciphering with HDFS chunk partitioning. - In **Project #8**, we showed how enterprise PaaS dramatically accelerates business application development through declarative data modeling, automated business rules, and pre-built analytical reporting engines.

Future Enhancements

1. **Physical Multi-Node Expansion**: Scaling the HDFS cluster across multiple physical nodes or Raspberry Pi edge boards with a replication factor of 3.
2. **Asymmetric Key Exchange**: Implementing RSA / hybrid cryptography so that chunk keys are encrypted under recipient public keys.
3. **Salesforce Experience Cloud Integration**: Publishing a public-facing Community portal enabling prospective students to directly submit their own applications and view admissions status in real time.

---

8. References & Technical Specifications

1. **Apache Software Foundation**. *Hadoop Distributed File System (HDFS) Architecture Guide*, Hadoop 3.3.6 Documentation, 2023. https://hadoop.apache.org/docs/r3.3.6/
2. **Apache Software Foundation**. *WebHDFS REST API Specification*, Hadoop 3.3.6 Documentation, 2023.
3. **National Institute of Standards and Technology (NIST)**. *FIPS PUB 197: Advanced Encryption Standard (AES)*, U.S. Department of Commerce, 2001.
4. **Krawczyk, H., Bellare, M., & Canetti, R.** *RFC 2104: HMAC: Keyed-Hashing for Message Authentication*, Internet Engineering Task Force (IETF), 1997.
5. **Python Cryptography Authority**. *Fernet Symmetric Encryption Specification*, 2024. https://cryptography.io/en/latest/fernet/
6. **Salesforce Inc.** *Salesforce Lightning Platform Fundamentals: An Introduction to Customizing Salesforce*, Salesforce Documentation, 2026.
7. **Salesforce Inc.** *Salesforce Lightning Design System (SLDS)*, 2026. https://www.lightningdesignsystem.com/